

WeUP Phase 0 — UI/UX Bible (Short Whitepaper ToC)

A concise, implementation-oriented table of contents. Each section is scoped to be **actionable by design and frontend teams**.

0. Abstract

- Product intent: **map-first nightlife radar**
 - Phase 0 scope: **Map + Upload (OCR) + Calendar + Event Detail + Save**
 - Non-goals (Phase 0): full ticketing, deep social graph
-

1. Design Philosophy

- 1.1 Map as Primary Surface
 - 1.2 Single-Viewport Doctrine (no page scroll)
 - 1.3 One Active Mode Rule
 - 1.4 Layered OS vs. Screen-Based App
 - 1.5 Signal > Noise (attention economy)
 - 1.6 Cinematic Minimalism (black + neon system)
-

2. System Architecture (UX)

- 2.1 View Modes (Map, Calendar, Event, Upload, Saved, Profile)
 - 2.2 State Machine Overview (explicit transitions)
 - 2.3 Layer Model (L0–L5)
 - 2.4 Control Visibility Contract
 - 2.5 Interaction Channels (tap, swipe, drag, long-press)
-

3. Spatial & Layout Principles

- 3.1 Viewport Budgeting (vertical/horizontal allocation)
- 3.2 Safe Zones (thumb, gesture, system bars)
- 3.3 Edge Anchoring vs. Floating Controls
- 3.4 Density Tiers (ambient / focus / modal)
- 3.5 Negative Space as Signal Amplifier

4. Navigation Model

- 4.1 Bottom Navigation (roles, compression rules)
 - 4.2 Mode Entry/Exit Patterns
 - 4.3 Back/Close Semantics (global vs. local)
 - 4.4 Deep State Behavior (nav suppression in modal states)
 - 4.5 Gesture Navigation (horizontal mode switching where applicable)
-

5. Time System (Critical)

- 5.1 Time as Map Filter (not global state)
 - 5.2 Time Control Patterns (pill, tray, dock)
 - 5.3 Visibility Rules per Mode
 - 5.4 Default States (Now / Tonight / Weekend)
 - 5.5 Feedback (active window, map response)
-

6. Map Mode (Primary)

- 6.1 Map Rendering (dark theme, contrast rules)
 - 6.2 Signal Visualization (pins, clusters, heat)
 - 6.3 Event Preview (peek cards, hover states)
 - 6.4 Minimal HUD (status, time access, nav)
 - 6.5 Performance Constraints (query windowing, clustering)
-

7. Calendar Mode (Modal System)

- 7.1 Takeover Behavior (true modal)
 - 7.2 Temporal Navigation (date chips / segments)
 - 7.3 Masonry/Grid System (variable card sizing)
 - 7.4 Event Density & Grouping (time/zone/genre)
 - 7.5 Internal Scroll Region (contained, not page scroll)
 - 7.6 Entry/Exit Transitions
-

8. Event Detail Mode

- 8.1 Modal Chamber Concept (focus state)
 - 8.2 Content Hierarchy (hero → title → meta → actions)
 - 8.3 CTA Cluster (Save, Route, Share, External Link)
 - 8.4 Background Recession (blur/dim/parallax)
 - 8.5 Close/Return Patterns
 - 8.6 Data Integrity (parsed vs. corrected fields)
-

9. Upload / OCR / Parser Flow

- 9.1 Guided Steps (Upload → OCR → Parse → Edit → Preview → Save)
 - 9.2 Confidence & Error Surfacing
 - 9.3 Inline Editing UX (fast correction)
 - 9.4 Progress & Status (async processing)
 - 9.5 Post-Submit Paths (view on map/calendar, boost options)
-

10. Saved Mode

- 10.1 Collection Models (tonight / week / custom)
 - 10.2 Card Layout (compact vs. expanded)
 - 10.3 Quick Actions (unsave, route, open)
 - 10.4 Cross-Mode Jump (back to map or detail)
 - 10.5 Empty/First-Time States
-

11. Profile Mode

- 11.1 Identity & Signal (level, badges optional)
 - 11.2 Preferences (radar, notifications, genres)
 - 11.3 Content Ownership (uploads, saved)
 - 11.4 Mode Isolation (no irrelevant controls)
 - 11.5 Account & Privacy
-

12. Component System

- 12.1 Core Components (Nav, Time Pill, Cards, Modals)
- 12.2 Card Variants (event, featured, compact)
- 12.3 Inputs (upload, edit fields)

- 12.4 Overlays & Sheets (modal, tray, toast)
 - 12.5 Reusability & Props Contracts
-

13. Visual System

- 13.1 Color Tokens (black, charcoal, neon green, accents)
 - 13.2 Typography (display vs. mono/meta)
 - 13.3 Elevation & Materials (glass, blur, shadow)
 - 13.4 Iconography & Micro-labels
 - 13.5 Accessibility (contrast, legibility)
-

14. Motion System

- 14.1 Principles (calm, cinematic, directional)
 - 14.2 Mode Transitions (map ↔ calendar ↔ detail)
 - 14.3 Micro-interactions (tap, expand, hover)
 - 14.4 Performance Targets (fps, duration budgets)
 - 14.5 Reduced Motion Fallbacks
-

15. Data & Query Model (UX-facing)

- 15.1 Event Schema (required vs. optional)
 - 15.2 Time Windows & Geo Queries
 - 15.3 Deduplication & Merge UX
 - 15.4 Loading & Empty States
 - 15.5 Error Handling (network, parse failures)
-

16. Monetization Surfaces (Phase 0)

- 16.1 Featured/Boosted Events (visual treatment)
 - 16.2 Venue Claim & Profile Upgrades
 - 16.3 Non-intrusive Placement Rules
 - 16.4 Disclosure & Trust
-

17. Security & Moderation (UX Hooks)

- 17.1 Reporting Flows
 - 17.2 Content Flags & States
 - 17.3 Admin/Curation Entry Points
 - 17.4 Abuse Mitigation UX
-

18. Analytics & Instrumentation

- 18.1 Core Events (view, save, upload, open detail)
 - 18.2 Funnels (upload → publish, view → save)
 - 18.3 Heatmaps (time/geo interactions)
 - 18.4 Experimentation (A/B hooks)
-

19. Performance & Constraints

- 19.1 Rendering Budgets (map + overlays)
 - 19.2 Network Strategy (pagination, caching)
 - 19.3 Image Handling (thumbnails, compression)
 - 19.4 Realtime vs. Polling Decisions
-

20. Implementation Blueprint

- 20.1 File/Component Structure
 - 20.2 State Management (store + finite state machine)
 - 20.3 API Contracts
 - 20.4 Feature Flags (Phase 0 toggles)
 - 20.5 CI/CD & Environments
-

21. Phase 0 Acceptance Criteria

- 21.1 Mode Clarity (one dominant mode)
 - 21.2 No Viewport Overflow
 - 21.3 Control Visibility Compliance
 - 21.4 End-to-End Upload → Event → Discover Loop
 - 21.5 Performance & Error Thresholds
-

22. Roadmap Bridge (Phase 1+)

22.1 Affiliate Ticketing Integration

22.2 Promoter Tools

22.3 Insights Dashboard

22.4 Social/Network Features (deferred)

Appendices

A. Z-Layer Contract (L0–L5 spec)

B. Component Inventory (props/events)

C. Copy & Microcopy Guidelines

D. Design Tokens (JSON)

E. Interaction Matrix (mode × control visibility)

WeUP Phase 0 — UI/UX Bible (Sections 0–2)

Version: 0.1 (Phase 0 Implementation Draft)

Scope: Foundational design + system architecture

0. Abstract

0.1 Product Intent

WeUP Phase 0 is a **map-first nightlife radar system**, not a traditional app.

It is designed to:

- Surface **real-time cultural activity**
- Convert **flyers** → **structured events** → **spatial signals**
- Allow users to **scan a city as a living system**

Core loop:

Upload → Extract → Parse → Place → Discover → Save

0.2 Phase 0 Functional Scope

The MVP includes:

- **Map Surface (Primary Interface)**
- **Flyer Upload Pipeline**
- **OCR + Event Parsing**
- **Calendar Exploration Layer**
- **Event Detail Modal**
- **Save / Shortlist System**

All features must operate within:

A single viewport, layered interaction system

0.3 Non-Goals (Phase 0)

Explicit exclusions to avoid system bloat:

- Native ticketing infrastructure
- Complex social networking
- Messaging systems
- Deep recommendation engines
- Full promoter dashboards

These are deferred to Phase 1+.

0.4 Core UX Principle

The system is not a collection of screens.
It is a **spatial operating layer over a city**.

Users do not “navigate pages.”
They **move through states of focus**.

0.5 Experience Target

The interface should feel like:

- A **live cultural radar**
- A **nightlife command surface**
- A **signal scanner for real-world activity**

It should not feel like:

- A CRUD dashboard
 - A ticketing website
 - A social feed
 - A generic mobile app
-

0.6 Success Criteria (Phase 0)

A successful Phase 0 delivers:

- Immediate understanding (≤ 5 seconds)
 - Clear mode awareness at all times
 - Zero viewport overflow
 - Smooth upload $\rightarrow$ discover loop
 - High visual clarity under density
-

1. Design Philosophy

1.1 Map as Primary Surface

The map is:

- The **default state**
- The **main interaction layer**
- The **source of truth for discovery**

All other features must:

- Attach to the map

- Emerge from the map
 - Return to the map
-

1.2 Single-Viewport Doctrine

Strict rule:

The entire application must fit within one viewport at all times.

Implications:

- No page-level scrolling
 - Only **contained scrolling inside modal regions (if necessary)**
 - All transitions occur via:
 - overlays
 - modals
 - state changes
-

1.3 One Active Mode Rule

At any moment:

Only one mode may dominate user attention.

Examples:

Mode	Dominant Surface
Map	Map
Calendar	Calendar
Event Detail	Event modal
Upload	Upload flow

Violation of this rule causes:

- cognitive overload
- visual crowding
- interaction ambiguity

1.4 Layered OS Model (Not Screen-Based)

WeUP behaves like an operating system:

- Layers stack intentionally
- Depth communicates priority
- Background recedes when focus increases

Not:

- page → page navigation
 - tabbed app switching
-

1.5 Signal Over Noise

Every element must justify its presence.

Remove:

- redundant controls
- persistent elements without context
- duplicated filters

Prioritize:

- clarity
 - focus
 - immediate comprehension
-

1.6 Cinematic Minimalism

Visual identity:

- Deep black base
- Neon green signal accents
- Glass + blur materials
- Strong typography

But:

Minimalism is enforced through **control reduction**, not visual stripping.

1.7 Contextual UI (Adaptive Visibility)

UI elements must appear only when relevant.

Examples:

- Time slider → only in Map Mode
 - Bottom nav → suppressed in deep modal states
 - Filters → contextual, not global
-

1.8 Spatial Breathing Room

The interface must:

- Avoid vertical compression
- Avoid stacking bars (nav + slider + cards)
- Preserve negative space

Result:

The user feels they are scanning a city, not reading a screen.

1.9 Deterministic Interaction

Every interaction must be:

- predictable
- reversible
- visually explained

No hidden states.

No ambiguous gestures.

2. System Architecture (UX)

2.1 View Modes (Explicit System States)

The system is composed of discrete modes:

2.1.1 Map Mode (Default)

Purpose: Exploration

- Full map visibility
 - Minimal UI
 - Time filtering active
 - Event signals visible
-

2.1.2 Calendar Mode

Purpose: Temporal browsing

- Full modal takeover
 - Dense event grid / masonry layout
 - Date-driven navigation
-

2.1.3 Event Detail Mode

Purpose: Focused event inspection

- Immersive modal
 - Strong content hierarchy
 - Clear CTA cluster
-

2.1.4 Upload Mode

Purpose: Data intake pipeline

- Guided multi-step flow

- OCR + parsing
 - Manual correction
-

2.1.5 Saved Mode

Purpose: Personal curation

- Event collection view
 - Quick access to saved signals
-

2.1.6 Profile Mode

Purpose: Identity & preferences

- Account management
 - Settings
 - No discovery clutter
-

2.2 State Machine Overview

The system must behave as a **finite state machine (FSM)**.

Core transitions:

MAP → EVENT

MAP → CALENDAR

MAP → UPLOAD

MAP → SAVED

MAP → PROFILE

CALENDAR → EVENT

EVENT → MAP

UPLOAD → MAP

SAVED → EVENT → MAP

Rules:

- No ambiguous overlapping states

- No dual-active modes
 - Every state has a clear entry and exit
-

2.3 Layer Model (Z-Depth System)

Define strict visual layers:

Layer 0 — World Surface

- Map
 - Background environment
-

Layer 1 — Context HUD

- Minimal status indicators
 - Mode labels
 - Map signals
-

Layer 2 — Controls

- Time control
 - Filters
 - Navigation
-

Layer 3 — Mode Containers

- Calendar modal
 - Upload flow
 - Saved view
-

Layer 4 — Focused Content

- Event detail modal

- Expanded cards
-

Layer 5 — System Dialogs

- Confirmations
 - Errors
 - OCR correction overlays
-

Rule:

Higher layer = higher user focus

2.4 Control Visibility Contract

Every control must declare:

- which mode it belongs to
- when it is visible
- when it is hidden

Example:

Control	Map	Calendar	Event	Upload	Profile
Time Slider	Yes	Optional	No	No	No
Bottom Nav	Yes	Conditional	Hidden	Hidden	Yes
Filters	Yes	Yes	No	No	No

2.5 Interaction Channels

Define all input types:

Tap

- Primary interaction
 - Select, open, confirm
-

Swipe

- Mode transitions (if used)
 - Modal dismissal (controlled)
-

Drag

- Time slider interaction
 - Map movement
-

Long Press (Optional)

- Secondary actions
 - Save shortcuts
-

Gesture Rules

- One gesture = one outcome
 - No overlapping gesture meanings
 - Clear visual feedback required
-

2.6 System Integrity Rules

To prevent regression into the current MVP issues:

1. No persistent global controls without justification
 2. No overlapping primary surfaces
 3. No stacked vertical bars competing for space
 4. No ambiguous modal states
 5. No hidden system logic
-

Closing Statement (Sections 0–2)

WeUP Phase 0 is not a UI problem—it is a **state, space, and priority problem**.

This architecture establishes:

- clarity of mode
- clarity of depth
- clarity of control
- clarity of interaction

Everything that follows (components, visuals, motion) must obey this system.

Next section options:

- **3–5 (Spatial + Navigation + Time System)** → critical for fixing your current UI issues
- or
- **Full implementation blueprint (Vue + Firebase aligned)**

WeUP Phase 0 — UI/UX Bible (Sections 3–5)

Focus: Spatial system, navigation logic, and time architecture

Objective: Eliminate cramped UI, resolve control conflicts, enforce clarity

3. Spatial & Layout Principles

3.1 Viewport Budgeting

The viewport is a **finite resource**.

You must explicitly allocate vertical space:

Zone	Max Height	Purpose
Top HUD	8–12%	Branding + status

Map Canvas	70–80%	Primary interaction
Bottom Zone	12–18%	Navigation OR contextual control

Rule:

Only one bottom element may occupy the bottom zone at a time.

This directly fixes:

- time slider vs nav collision
- stacked bottom UI clutter

3.2 Safe Zones (Thumb + System)

Design for **one-handed operation**:

- Bottom 25% = primary interaction zone
- Avoid placing **conflicting controls in the same thumb arc**
- Floating elements must not overlap gesture zones

Constraint:

- Upload button must not block:
 - nav taps
 - slider interaction
 - modal expansion gestures

3.3 Edge Anchoring vs Floating Controls

Edge Anchored (Preferred)

- Bottom navigation
- Time access trigger
- Close/back controls

Floating (Restricted)

- Only for:

- contextual quick actions
- temporary states

Rule:

Floating controls must disappear when focus increases.

3.4 Density Tiers

Define UI density explicitly:

Tier 0 — Ambient (Map Mode)

- Minimal UI
 - Maximum breathing room
 - Signals over controls
-

Tier 1 — Informational (Calendar / Saved)

- Moderate density
 - Structured grid
 - Controlled scroll region
-

Tier 2 — Focus (Event Detail / Upload)

- High information density
 - Fully intentional layout
 - No background competition
-

Rule:

Density must increase only when focus increases.

3.5 Negative Space as Signal Amplifier

Negative space is not empty—it is **functional clarity**.

Remove:

- redundant bars
- stacked overlays
- unnecessary labels

Preserve:

- spacing between cards
 - margin around modals
 - map visibility
-

3.6 Vertical Conflict Elimination

The current MVP problem:

[Top Bar]
[Map]
[Event Card]
[Time Slider]
[Bottom Nav]

This is invalid.

Correct model:

Map Mode:

[Top HUD]
[Map Canvas]
[Bottom Nav]

Time Active (Expanded):

[Top HUD]
[Map Canvas]
[Time Tray replaces Bottom Nav]

3.7 Modal Containment

All modal systems must:

- Stay within viewport
- Not overflow edges
- Use internal scroll if needed

Rule:

The page never scrolls. Only modal content can scroll internally.

3.8 Spatial Priority Rule

At any moment:

1. One dominant surface
 2. One supporting control layer
 3. Everything else recedes
-

4. Navigation Model

4.1 Navigation Philosophy

Navigation is not tab switching.

It is:

Mode invocation + state transition

4.2 Bottom Navigation (Rebuilt)

Structure (5 items max):

- Map (default)
 - Calendar
 - Upload (center emphasis)
 - Saved
 - Profile
-

Behavior Rules:

In Map Mode

- Fully visible
- Primary navigation surface

In Calendar Mode

- Either:
 - fades (reduced opacity), or
 - is replaced by local calendar controls

In Event Detail Mode

- Hidden OR replaced with minimal back/close

In Upload Mode

- Hidden (focus required)

In Profile Mode

- Visible
-

Rule:

Bottom nav must never compete with modal or time controls.

4.3 Mode Entry / Exit

Each mode must have:

- Clear entry trigger
- Predictable exit behavior

Entry Examples:

Mode	Entry
Calendar	Bottom nav tap
Event	Map pin or card tap
Upload	Center nav button
Saved	Nav tap
Profile	Nav tap

Exit Rules:

Mode	Exit Behavior
Event	Close → return to previous state
Calendar	Close → return to map
Upload	Cancel or complete → map
Saved	Back → map

4.4 Back vs Close Semantics

Back

- Returns to previous mode
- Preserves context

Close

- Exits modal

- Resets to base state
-

Rule:

Do not mix back and close randomly.

4.5 Gesture Navigation

Allowed gestures:

Swipe Down

- Dismiss modal (Event / Calendar)

Swipe Horizontal (Optional)

- Switch between modes (only if consistent)

Drag

- Map movement
 - Time adjustment
-

Forbidden:

- Gestures with multiple meanings
 - Hidden gesture-only navigation
-

4.6 Deep State Behavior

When entering high-focus modes:

- Bottom nav → suppressed
- Map → backgrounded
- Time controls → hidden

Rule:

The deeper the state, the fewer global controls remain.

4.7 Navigation Conflict Resolution

If two controls compete:

1. Keep the one tied to the active mode
 2. Hide or collapse the other
-

5. Time System (Critical Fix Area)

5.1 Time as Contextual Filter

Time is:

A map exploration parameter

It is NOT:

- global state
 - always visible control
 - persistent UI bar
-

5.2 Time Visibility Rules

Mode	Time Visibility
Map	Visible or accessible
Calendar	Integrated or optional

Event	Hidden
Upload	Hidden
Saved	Hidden
Profile	Hidden

5.3 Time Control Patterns (Redesign)

You must choose ONE primary pattern.

Option A — Time Pill (Recommended)

- Small pill in bottom corner
- Shows:
 - “Now”
 - “Tonight”
 - time range

Tap → expands into time tray

Option B — Time Tray

- Slides up from bottom
 - Temporarily replaces nav
 - Allows:
 - drag
 - range selection
-

Option C — Time Dock

- Floating mini bar
 - Expandable on interaction
-

Rule:

Time control must not occupy permanent vertical space.

5.4 Default Time States

Provide quick presets:

- Now
 - Tonight
 - Weekend
 - Custom
-

5.5 Time Feedback System

When time changes:

- Map updates immediately
- Pins animate (fade/shift)
- Heatmap adjusts

User must feel:

“I am scanning a different time layer of the city”

5.6 Time + Map Coupling

Time must only affect:

- map signals
- event visibility

It must NOT affect:

- profile
 - upload
 - saved collections (unless explicitly filtered)
-

5.7 Time Conflict Elimination

Remove:

- global time bars
 - duplicate time indicators
 - time appearing in unrelated modes
-

5.8 Time Interaction Model

Interaction Flow:

1. User taps time pill
 2. Time tray expands
 3. User adjusts
 4. Map updates live
 5. Tray collapses
-

5.9 Time Persistence

Time selection persists:

- within Map Mode session
- optionally across sessions

But:

must always be visible via pill indicator

5.10 System Integrity Rule

If time is visible, it must be actionable.

If it is not actionable, it must not be visible.

Closing Statement (Sections 3–5)

These sections resolve the core MVP issues:

- **Cramped layout** → fixed via viewport budgeting
- **Control collisions** → fixed via visibility contracts
- **Time slider problem** → fixed via contextual system
- **Navigation confusion** → fixed via mode-based logic

The system now enforces:

- one dominant surface
 - one active mode
 - one clear interaction layer
-

Next Sections Recommended

- **6–8 (Map, Calendar, Event Detail redesign)** → where your product becomes visually differentiated
- or
- **Full Firebase + Vue implementation mapping of these rules**

WeUP Phase 0 — UI/UX Bible (Sections 6–8)

Focus: Map Mode, Calendar Mode, Event Detail Mode

Objective: Define the three most important experiential surfaces in the product

6. Map Mode (Primary Surface)

6.1 Purpose

Map Mode is the **default operational state** of WeUP.

It is where the user:

- scans the city
- reads cultural density
- filters by time
- opens event signals
- decides where to go next

This is not a dashboard.

This is a **live spatial discovery surface**.

6.2 Experience Goal

Map Mode should feel like:

- nightlife radar
- cultural heat scanner
- city signal field
- real-time urban interface

It should not feel like:

- event list over a map
 - rideshare map clone
 - cluttered GIS screen
 - admin monitoring tool
-

6.3 Dominant Surface Rule

In Map Mode:

The map must occupy the overwhelming majority of perceptual attention.

This means:

- no large persistent cards
 - no globally expanded drawers
 - no constant modal stack
 - no permanent time bar
 - no visual obstruction of the city surface
-

6.4 Layout Structure

Top HUD

Contains only:

- WeUP wordmark / compact identity
- optional city label
- small status pill such as:
 - Live
 - Tonight
 - Active Signals

This region must remain compact.

Main Canvas

Contains:

- dark city map
- event signals
- cluster glows
- optional subtle district overlays
- optional live pulse effects

This is the hero zone.

Bottom Zone

Contains:

- bottom nav in default state
 - OR time tray when time is expanded
 - never both fully active at the same time
-

6.5 Signal Visualization System

Event representation must be layered by importance.

Signal Types

Micro Signals

- small neon dots or tiny glow points
- used for lower-confidence or lower-priority events

Standard Signals

- medium glowing pins
- default event marker

Featured Signals

- brighter ringed pins
- boosted or important events

Cluster Signals

- grouped pulse nodes
 - show event count and intensity
-

6.6 Visual Behavior of Signals

Signals should:

- glow subtly at rest
- brighten on proximity or tap
- pulse lightly when active in selected time range

They should not:

- bounce excessively
 - obscure map readability
 - use too many competing colors
-

6.7 Event Preview Pattern

Map Mode should not display full event cards by default.

Instead use:

Peek Card Pattern

When a signal is tapped:

- a compact preview card rises from the lower portion of the map
- shows:
 - title
 - venue
 - time
 - one category tag
- includes:
 - open detail
 - quick save

The peek card must remain compact.

6.8 Peek Card Rules

Peek card may:

- occupy only a small percentage of vertical space
- collapse easily
- disappear if another signal is selected

Peek card may not:

- become a giant permanent drawer
 - compete with the map as a primary layer
-

6.9 Time System in Map Mode

Map Mode is the primary home of time.

Recommended implementation

- compact time pill in lower right or lower center
- tap expands into time tray
- tray temporarily replaces nav prominence

The time system must feel like a **city-layer switcher**.

6.10 Filters in Map Mode

Filters must remain minimal.

Recommended:

- genre filter chip
- neighborhood filter chip
- maybe one “live now” toggle

Do not expose full filter complexity by default.

Advanced filtering should be expandable, not persistent.

6.11 Motion in Map Mode

Transitions should be subtle:

- map remains stable
- selected signal slightly enlarges
- preview card lifts softly
- time shift animates signal fade/repopulation

The map should feel alive, not chaotic.

6.12 Empty / Sparse States

If few events exist:

- show softer, more spacious map
- include prompt:
 - “No major signals in this time layer”
 - “Try Tonight” or “Expand area”

Sparse state must still feel premium.

6.13 Performance Rule

Map Mode must remain fast under density.

Requirements:

- clustering at scale
 - viewport query bounding
 - lazy-loading detail metadata
 - reduced effect density when many signals exist
-

7. Calendar Mode (Modal Temporal Surface)

7.1 Purpose

Calendar Mode is the **temporal exploration chamber**.

It is where the user:

- browses events by day/time
- compares density across time periods
- scans grouped event tiles
- opens detail intentionally

It is not merely a tab with a list.

7.2 Experience Goal

Calendar Mode should feel like:

- cultural mission board
- nightlife signal grid
- premium event matrix
- temporal intelligence wall

It should not feel like:

- plain list of cards
 - generic event feed
 - standard mobile calendar clone
-

7.3 Mode Behavior

Calendar Mode must behave as a **true takeover layer**.

That means:

- the calendar becomes the dominant surface
 - map recedes into atmospheric background only if retained at all
 - time controls from map do not remain as global bars
 - bottom nav is reduced or replaced by local calendar navigation
-

7.4 Entry Pattern

Entry occurs through:

- bottom nav tap
- optional upward gesture from map mode

Transition should feel like:

rising from spatial scan into temporal board

7.5 Layout Structure

Header Zone

Contains:

- large Calendar title
- active date context
- date chips / segmented selector
- close/back affordance

Main Calendar Body

Contains:

- masonry-style modular event grid
- internally scrollable content region if needed
- grouped event cards

Lower Utility Zone

Optional:

- local mode controls
 - sort or density toggle
 - not the global bottom nav competing at full weight
-

7.6 Date Navigation

Recommended date controls:

- Today
- Tomorrow
- This Weekend
- custom date chips

Or:

- horizontally scrollable day strip

The active day must be extremely obvious.

7.7 Calendar Content Model

This is not a literal month calendar.

This is a **cultural density calendar**.

Recommended structure:

- daily signal grouping
- modular tiles by relevance
- larger tiles for:
 - featured events

- near-term events
 - highly saved events
 - smaller tiles for supporting events
-

7.8 Masonry Logic

Cards may vary by:

- feature priority
- image richness
- time relevance
- event confidence
- popularity or save count

This creates a living, editorialized event board rather than a flat list.

7.9 Card Information Hierarchy

Each calendar card should include:

Required

- event title
- time
- venue or neighborhood
- small category tag

Optional

- flyer thumbnail
- signal intensity
- featured badge

Do not overload every tile.

7.10 Calendar Scroll Policy

The page itself does not scroll.

Only the calendar content region may scroll internally if necessary.

This preserves the single-viewport doctrine.

7.11 Calendar Selection Behavior

When a user taps a card:

- the card expands or transitions into Event Detail Mode
- background calendar recedes
- chosen event becomes dominant

This must feel deliberate and premium.

7.12 Calendar Density Controls

Optional local controls:

- sort by time
- sort by relevance
- sort by neighborhood

These controls must be lightweight and local to calendar mode.

7.13 Background Treatment

The background behind calendar may be:

- dimmed map
- blurred city texture
- subtle atmospheric lighting

It must not visually compete with the calendar board.

7.14 Exit Behavior

Closing calendar returns the user to:

- prior map state
- preserved time context where relevant

This continuity is important.

8. Event Detail Mode (Focused Cultural Object)

8.1 Purpose

Event Detail Mode is the **focus chamber** for one nightlife object.

It is where the user:

- understands the event clearly
- decides whether to save, route, share, or attend
- sees parsed flyer information in a readable structure

This is the most concentrated informational state in Phase 0.

8.2 Experience Goal

Event Detail should feel like:

- entering the signal
- spotlighting one cultural object
- premium event capsule
- immersive but readable modal chamber

It should not feel like:

- a cramped card expansion
- a generic info sheet
- a cluttered popup over an active app

8.3 Mode Behavior

Event Detail Mode is a **high-focus modal state**.

When entered:

- time slider hides
- global filters hide
- bottom nav hides or becomes irrelevant
- background recedes strongly

This mode owns the user's attention.

8.4 Entry Sources

Entry can happen from:

- map signal tap → open detail
- calendar tile tap → open detail
- saved item tap → open detail
- post-upload preview → open detail

Regardless of entry source, the event detail experience must be consistent.

8.5 Layout Structure

Hero Region

Contains:

- flyer image or visual header
- event title
- optional featured or confidence badge

Metadata Region

Contains:

- venue
- date
- time
- neighborhood
- genre tags
- source attribution if useful

Description / Context Region

Contains:

- parsed text summary
- notes
- possibly address or venue details

CTA Region

Contains:

- Save
 - Route / Map
 - Share
 - external link or RSVP placeholder
-

8.6 Hero Region Rules

The hero region should:

- establish emotional tone
- use flyer or branded visual crop
- avoid being too tall on small devices

It must leave enough room for real information.

8.7 Metadata Grouping

Information must be grouped by function, not dumped linearly.

Recommended grouping:

Time Block

- day
- start time
- end time if known

Place Block

- venue
- address or neighborhood

Identity Block

- genre
 - event type
 - source
-

8.8 CTA Hierarchy

Actions must be prioritized.

Primary CTA

- Save
- or
- Route

Secondary CTA

- Share
- Open external link

Do not show too many equal-weight actions.

8.9 Background Recession

When event detail is open:

- background map/calendar blurs or dims
- no unrelated motion continues

- background remains atmospheric only

The user should feel:

I have stepped into the event

8.10 Modal Shape and Containment

Event detail may be:

- center chamber modal
- bottom-origin immersive sheet
- nearly full-screen card

But it must:

- respect viewport
 - allow internal scrolling if content exceeds space
 - never create page overflow
-

8.11 Parsed Data Integrity UX

Because data may come from OCR and parsing, the UI must gracefully handle uncertainty.

Display:

- strong confidence data normally
- weak confidence data more cautiously
- optional parsed / corrected distinction if useful

Bad data should not visually break trust.

8.12 Missing Data Handling

If some fields are missing:

- preserve clean layout
- hide absent rows entirely

- avoid empty placeholders

Example:

- if no address, do not render an empty address block
-

8.13 Exit Behavior

Closing detail returns the user to the source context:

- back to selected map region
- back to same calendar position
- back to saved list state

Do not dump the user into a reset state unless explicitly intended.

8.14 Motion Behavior

Recommended transition:

- source card or pin subtly enlarges
- event chamber rises and sharpens
- background recedes with blur/dim
- close returns smoothly to source

This preserves spatial memory.

8.15 Trust and Premium Feel

Event Detail Mode is where perceived product quality is decided.

If this surface is:

- clear
- spacious
- cinematic
- legible

then the entire product feels intentional.

If it is cluttered, everything feels unfinished.

Closing Statement (Sections 6–8)

These three sections define the core experiential architecture:

- **Map Mode** = spatial discovery
- **Calendar Mode** = temporal exploration
- **Event Detail Mode** = focused cultural inspection

Together, they create the primary discovery loop:

Scan city → inspect time layer → open event → save or route

This is the operational heart of WeUP Phase 0.

Next recommended sections

- **9–11:** Upload / OCR / Parser, Saved, Profile
- **12–14:** Component system, visual system, motion system

WeUP Phase 0 — UI/UX Bible (Sections 9–11)

Focus: Upload Pipeline, Saved System, Profile Mode

Objective: Complete the core user loop + ensure system cohesion beyond discovery

9. Upload / OCR / Parser Flow (Data Intake Engine)

9.1 Purpose

The upload system is the **supply engine** of WeUP.

It converts:

Flyer → OCR → Structured Event → Map Signal

This is not just a form.

It is a:

guided cultural ingestion pipeline

9.2 Experience Goal

The upload flow should feel like:

- a premium intake process
- intelligent parsing system
- fast correction interface
- creator-friendly tool

It should not feel like:

- a generic upload form
 - a bureaucratic input screen
 - a cluttered multi-field UI
-

9.3 Mode Behavior

Upload Mode is a **high-focus guided flow**.

When active:

- bottom nav → hidden
- map → backgrounded or removed
- time controls → hidden
- only upload-relevant UI remains

9.4 Step-Based Flow Architecture

The flow must be broken into **clear sequential stages**:

Step 1 — Input

User:

- uploads flyer (camera or gallery)

UI:

- large drop zone or preview surface
 - immediate visual feedback
-

Step 2 — OCR Processing

System:

- extracts text via AI

UI:

- loading state with clear messaging:
 - “Extracting event details...”
 - subtle animation (not distracting)
-

Step 3 — Parsed Output

System presents structured data:

- title
- date
- time
- venue
- location
- tags/genre

- description
-

9.5 Confidence Layer

Each field should optionally show:

- high confidence → normal display
- medium confidence → subtle indicator
- low confidence → highlight or suggestion

Rule:

Do not overwhelm the user with technical confidence metrics.
Surface uncertainty only when actionable.

9.6 Step 4 — Manual Correction

User can:

- tap any field
- edit inline
- correct misread OCR data

UX Requirements:

- inline editing (not separate forms)
 - fast input
 - minimal friction
-

9.7 Editing Model

Each field behaves as:

- tap → editable state
- confirm → returns to structured layout

Avoid:

- full-page form transitions
 - multi-screen editing
-

9.8 Step 5 — Preview Integration

Before saving:

User sees:

- how event appears on map or calendar
- event card preview
- signal strength/visibility

This builds trust.

9.9 Step 6 — Save / Publish

User can:

- publish event
- optionally boost (future monetization hook)

Post-save:

- event enters system
 - visible on map/calendar
-

9.10 Error Handling

If OCR fails or is incomplete:

- show partial extraction
 - allow full manual entry fallback
 - never block submission entirely
-

9.11 Upload Exit Paths

After completion:

- go to event detail
 - return to map
 - optionally prompt:
 - “Boost visibility?”
-

9.12 Motion Behavior

Flow transitions must feel linear:

- step slides or fades
 - progress indicator updates
 - no abrupt context switches
-

9.13 System Integrity Rule

The upload flow must always feel faster than manual entry.

10. Saved Mode (Personal Signal Layer)

10.1 Purpose

Saved Mode is the user's:

- personal event memory
 - shortlist of intent
 - curated nightlife plan
-

10.2 Experience Goal

Saved Mode should feel like:

- a curated collection
- a personal radar log
- an organized plan

It should not feel like:

- a generic bookmark list
 - a cluttered archive
 - a secondary feature
-

10.3 Mode Behavior

Saved Mode is a **medium-focus browsing state**.

When active:

- map → backgrounded or removed
 - time slider → hidden
 - bottom nav → visible
-

10.4 Layout Structure

Header

- “Saved” or “My Nights”
- optional filter controls

Body

- event cards in:
 - grid
 - stacked layout
 - or grouped sections
-

10.5 Grouping Strategies

Saved events may be grouped by:

- Tonight
- This Weekend
- Upcoming
- Past (optional)

This gives structure immediately.

10.6 Card Design

Saved cards should be:

- compact
- consistent with calendar cards
- slightly richer than map preview cards

Include:

- title
 - time
 - venue
 - tag
-

10.7 Interaction Model

Each card supports:

- tap → open Event Detail
 - quick unsave (icon or swipe)
 - optional quick route
-

10.8 Empty State

If no saved events:

Show:

- “No saved signals yet”
- CTA: “Explore the map”

Must feel intentional, not broken.

10.9 Cross-Mode Continuity

Saved → Event → Map must preserve:

- spatial context
 - temporal context where applicable
-

10.10 Optional Enhancements (Phase 0+)

- reorder events manually
 - tag events
 - simple notes
-

10.11 System Integrity Rule

Saved Mode must reduce cognitive load, not recreate it.

11. Profile Mode (Identity Layer)

11.1 Purpose

Profile Mode is the user's:

- identity anchor
 - preference control center
 - system interaction summary
-

11.2 Experience Goal

Profile should feel like:

- a clean personal control space
- integrated with the product's aesthetic
- minimal and intentional

It should not feel like:

- a settings dump
 - a generic account page
 - a cluttered dashboard
-

11.3 Mode Behavior

Profile Mode is a **low-frequency but important state**.

When active:

- map → removed
 - time slider → hidden
 - bottom nav → visible
-

11.4 Layout Structure

Top Section

- user name
 - optional avatar
 - signal or activity indicator (optional)
-

Core Sections

Account

- email / login method
- logout

Preferences

- genres of interest
- notification settings
- time preferences (optional)

Activity

- uploaded events
 - saved events
-

11.5 Visual Language

Profile must:

- maintain dark + neon aesthetic
 - increase spacing compared to other modes
 - reduce density
-

11.6 Control Scope

Profile must not include:

- time slider
 - map filters
 - discovery UI
-

11.7 Navigation Behavior

Profile should:

- feel like a branch of the system
- not a completely separate app

Return path must be:

- clear
- fast
- predictable

11.8 Data Ownership

Users should be able to:

- see uploaded events
- edit or remove their content (if allowed)
- manage saved items

11.9 Privacy & Trust

Include:

- basic privacy controls
- minimal but clear information usage

11.10 System Integrity Rule

Profile Mode must be the cleanest, least cluttered space in the app.

Closing Statement (Sections 9–11)

These sections complete the full Phase 0 loop:

Upload → Parse → Map → Calendar → Event → Save → Return

- **Upload Mode** powers supply
- **Saved Mode** captures intent
- **Profile Mode** anchors identity

Together with Sections 6–8, the system now forms a complete, coherent UX architecture.

Next Sections Recommended

- **12–14** → Component system, visual system, motion system (critical for implementation)
- or
- Full **Vue + Firebase component + state implementation mapping**

WeUP Phase 0 — UI/UX Bible (Sections 12–14)

Focus: Component System, Visual System, Motion System

Objective: Convert the UX architecture into a buildable design language

12. Component System

12.1 Purpose

The component system exists to ensure:

- consistency
- speed of implementation
- predictable behavior
- reusable interaction patterns

Every component in WeUP must serve one of three roles:

1. **Navigation**
2. **Discovery**
3. **Focus / Input**

No component should exist without a defined role in the state system.

12.2 Component Categories

The WeUP component library should be organized into the following groups:

A. Shell Components

High-level layout and navigation surfaces.

- AppShell
 - TopHUD
 - BottomNav
 - ModeContainer
 - ModalLayer
 - BackdropLayer
-

B. Discovery Components

Map, calendar, event signal, and browse components.

- MapCanvas
 - SignalPin
 - SignalCluster
 - EventPeekCard
 - CalendarBoard
 - CalendarCard
 - FeaturedEventTile
 - FilterChip
 - TimePill
 - TimeTray
-

C. Focus Components

High-attention components used in modal and detail states.

- EventDetailModal
 - EventHero
 - MetadataBlock
 - CTACluster
 - UploadWizard
 - OCRStatusPanel
 - ParsedField
 - InlineEditField
 - SaveConfirmationDialog
-

D. Identity Components

User and personal collection surfaces.

- SavedGrid
 - SavedCard
 - EmptyStatePanel
 - ProfileHeader
 - PreferenceGroup
 - ActivityList
-

E. Feedback Components

Status and supporting system communication.

- LoadingState
 - Toast
 - InlineError
 - FieldConfidenceHint
 - ProgressStepper
 - ConfirmationSheet
-

12.3 Shell Component Contracts

AppShell

Owns:

- viewport sizing
- safe-area padding
- root layering
- mode switching container

Rules:

- no page scroll
 - full-height containment
 - all modes render inside shell
-

TopHUD

Contains:

- WeUP identity
- local mode label
- compact status indicators

Rules:

- always compact
 - never becomes a full toolbar
 - should adapt by mode
-

BottomNav

Contains:

- Map
- Calendar
- Upload
- Saved
- Profile

Rules:

- max 5 destinations
 - active icon state required
 - can suppress in focus-heavy modes
 - center upload affordance allowed, but not oversized
-

ModalLayer

Responsible for:

- calendar takeover
- event detail chamber
- upload flow
- system dialogs

Rules:

- centralized layering contract
 - no ad hoc modal stacking
-

12.4 Discovery Component Contracts

MapCanvas

Owns:

- map rendering
- signal placement
- viewport interactions
- spatial background continuity

Rules:

- always fast
 - never blocked by unnecessary overlays
 - supports cluster reduction
-

SignalPin

Represents one event.

Props should support:

- priority
- featured state
- selected state
- time relevance
- save status

Rules:

- minimal idle motion
 - clear selected state
 - accessible hitbox
-

SignalCluster

Represents grouped events.

Should show:

- count
- intensity
- optional glow scaling

Rules:

- count readable
 - tap expands cluster region or zooms intelligently
-

EventPeekCard

Purpose:

- quick insight without leaving map mode

Contains:

- title
- venue
- time
- one tag
- save shortcut
- open detail action

Rules:

- compact only
 - must not become a giant sheet
 - easy dismissal
-

CalendarBoard

Owns:

- date grouping
- masonry tile composition
- internal scroll region
- local calendar controls

Rules:

- must behave as a destination surface
 - no generic list rendering
-

CalendarCard / FeaturedEventTile

Support variants:

- compact
- standard
- featured

Rules:

- consistent metadata hierarchy
 - image optional, not required
 - featured state visually distinct but controlled
-

FilterChip

Use for:

- genre
- neighborhood
- day presets
- relevance toggles

Rules:

- do not overpopulate the UI
 - only show contextual filters
-

TimePill / TimeTray

Together they form the time system.

TimePill

- collapsed access point

- visible only when relevant

TimeTray

- expanded selector
- replaces or suppresses nav while active

Rules:

- no persistent full-width slider bar
 - time must be contextual and lightweight
-

12.5 Focus Component Contracts

EventDetailModal

Owns:

- focused event reading
- CTA actions
- trust layer for OCR-derived content

Rules:

- no unrelated global controls
 - supports internal scroll
 - must preserve source context on exit
-

EventHero

Contains:

- flyer image
- title
- event tone

Rules:

- visually strong
- height constrained
- never consumes too much viewport

MetadataBlock

Variants:

- TimeBlock
- PlaceBlock
- IdentityBlock

Rules:

- semantic grouping required
 - hide missing fields cleanly
-

CTACluster

Primary actions:

- Save
- Route

Secondary actions:

- Share
- External link

Rules:

- primary CTA must be visually dominant
 - avoid too many equal-priority actions
-

UploadWizard

Owns:

- step sequence
- upload progression
- correction flow

Rules:

- clear steps

- no mode confusion
 - minimal distraction
-

OCRStatusPanel

Used during OCR / parsing.

Contains:

- progress
- system status
- helpful messaging

Rules:

- reassuring but concise
 - avoid technical overload
-

ParsedField / InlineEditField

Purpose:

- display extracted event data
- enable correction

Rules:

- editing must be fast
 - confidence hint subtle
 - input state visually obvious
-

12.6 Identity Component Contracts

SavedGrid / SavedCard

Purpose:

- saved event management
- cross-mode return

Rules:

- compact browsing
 - fast open / unsave
 - grouped organization
-

EmptyStatePanel

Used in:

- Saved
- sparse map states
- no uploads

Rules:

- always intentional
 - always action-oriented
 - never dead or generic
-

ProfileHeader / PreferenceGroup / ActivityList

Purpose:

- account summary
- preference management
- uploaded/saved activity visibility

Rules:

- generous spacing
 - strong readability
 - low clutter
-

12.7 Feedback Component Contracts

LoadingState

Must exist for:

- map data fetch
- OCR processing
- event save actions
- route fetches

Rules:

- branded but restrained
 - no blocking unless necessary
-

Toast

Use for:

- save confirmed
- upload completed
- unsave completed
- minor errors

Rules:

- short-lived
 - non-obstructive
 - bottom-safe placement
-

InlineError

Use for:

- OCR missing fields
- bad data
- invalid input

Rules:

- local to the problem
 - avoid generic banners whenever possible
-

12.8 Component Behavior Rules

Every reusable component must define:

- state variants
- visible modes
- transitions
- entry/exit patterns
- fallback behavior

No component should be visually designed without a behavioral contract.

12.9 Component Composition Principles

1. Small components compose into larger mode surfaces
 2. Large components do not own unrelated concerns
 3. Modal components do not render global controls internally
 4. Reusability must not destroy mode clarity
-

12.10 System Integrity Rule

A component is valid only if its role, layer, mode scope, and transition behavior are explicitly defined.

13. Visual System

13.1 Purpose

The visual system translates WeUP's product identity into a consistent buildable language.

It must create:

- premium nightlife atmosphere
 - high clarity under density
 - strong brand distinction
 - readability under dark UI conditions
-

13.2 Visual Identity Principle

WeUP's interface should feel like:

- city radar after dark
- premium nightlife console
- cinematic urban scanner
- modern cultural signal operating layer

The style must never drift into:

- gamer UI excess
 - generic SaaS dashboard
 - overdecorated cyberpunk parody
 - flat default mobile patterns
-

13.3 Color System

Core Base Colors

- Deep black
- Charcoal
- Smoked graphite
- muted dark gray

These form the environmental substrate.

Accent Colors

Primary accent:

- neon green / signal green

Optional support accents:

- soft white for primary text
 - muted gray for metadata
 - restrained amber only for rare highlight or warning states
-

Usage Rules

- neon green is for signal, action, selection, emphasis
 - never flood the interface with accent color
 - dark space must remain dominant
-

13.4 Material System

WeUP should use a layered material approach:

Base Material

- matte black / charcoal background planes

Glass Material

- translucent panels
- subtle blur
- low-opacity borders

Signal Material

- glowing rings
 - highlighted chips
 - pin illumination
-

Rules

- glass is used to separate layers, not decorate everything
 - blur should support focus hierarchy
 - shadows must remain soft and low-key
-

13.5 Typography System

Typography must separate:

- dramatic display language
- technical metadata

- compact utility labels
-

Display Type

Use for:

- mode headers
- event titles
- key branded surfaces

Characteristics:

- bold
 - slightly extended or italicized
 - sharp and cinematic
-

UI Text

Use for:

- labels
- buttons
- section headers

Characteristics:

- clean
 - modern
 - highly readable
-

Metadata Text

Use for:

- tags
- timestamps
- source references
- confidence hints

Characteristics:

- compact
 - technical
 - optionally monospaced or semi-mono in select surfaces
-

13.6 Typography Hierarchy Rules

There must be clear separation between:

1. Title
2. Supporting information
3. Metadata
4. Action labels

No surface should make these levels visually ambiguous.

13.7 Shape Language

WeUP's shape language should balance:

- soft glass rounding
- disciplined geometry
- urban precision

Recommended:

- rounded corners for modal surfaces
 - sharper geometry for map signals and chips
 - consistent radius system across components
-

13.8 Iconography

Icons must be:

- simple
- legible at small sizes
- slightly technical in tone

Do not use overly playful icon language.

13.9 Borders, Dividers, and Strokes

Use borders sparingly.

Preferred:

- subtle edge light
- low-opacity divider lines
- gentle elevation difference

Avoid:

- heavy outlines
 - visually noisy separators
-

13.10 Image Handling

Flyer imagery is important but dangerous to layout consistency.

Rules:

- crop intentionally
 - never allow image chaos to break the system
 - maintain dark overlay when necessary
 - preserve enough of the flyer to communicate identity
-

13.11 Data Density and Readability

The visual system must survive both:

- sparse states
- dense nightlife signal states

This means:

- spacing scales intelligently
- metadata stays legible
- accent use remains controlled

13.12 Accessibility

Even with a nightlife aesthetic, the UI must maintain:

- sufficient contrast
- readable text sizes
- clear touch targets
- visible selection states

Dark mode aesthetics do not excuse poor legibility.

13.13 Visual Hierarchy Corrections

To fix the current MVP issues:

- reduce the number of simultaneous bright accents
 - avoid three competing bright focal points
 - increase card padding consistency
 - separate surfaces with depth, not clutter
 - ensure background recedes when foreground is active
-

13.14 System Integrity Rule

The visual system must amplify focus, not compete with it.

14. Motion System

14.1 Purpose

Motion in WeUP is not decorative.

It exists to communicate:

- mode change
 - layer transition
 - focus shift
 - system response
 - spatial continuity
-

14.2 Motion Philosophy

Motion should feel:

- cinematic
- composed
- premium
- directional
- urban
- fluid

It should not feel:

- bouncy
 - playful
 - chaotic
 - game-like
 - over-animated
-

14.3 Core Motion Roles

Motion must serve one of the following:

1. **Reveal**
 2. **Transition**
 3. **Confirm**
 4. **Recede**
 5. **Guide**
-

14.4 Reveal Motion

Used for:

- event peek card appearing
- modal entering
- time tray opening
- toasts appearing

Behavior:

- soft lift
 - fade in
 - slight sharpen or blur reduction
-

14.5 Transition Motion

Used for:

- Map → Calendar
- Calendar → Event
- Upload step progression
- Saved → Event

Behavior:

- preserve directional logic
 - maintain user orientation
 - avoid visual resets when possible
-

14.6 Confirm Motion

Used for:

- save action
- upload completion
- successful edit
- boost applied

Behavior:

- brief
- crisp
- localized

Examples:

- button state pulse
 - icon fill
 - small glow confirmation
-

14.7 Recede Motion

Used when background loses priority.

Examples:

- map blurs behind event detail
- bottom nav fades when time tray expands
- calendar dims when detail opens

Behavior:

- dim
 - blur
 - slight scale reduction if appropriate
-

14.8 Guide Motion

Used to:

- indicate draggable surfaces
- show expansion path
- establish source-to-destination relation

Example:

- tapped pin subtly links to detail modal rise
 - selected card expands into full detail chamber
-

14.9 Mode Transition Logic

Map → Calendar

- calendar rises or slides in as a controlled takeover
 - map remains faintly visible or fully replaced depending on chosen style
 - bottom nav recedes
-

Calendar → Event

- selected tile expands
 - surrounding tiles dim
 - detail chamber sharpens into focus
-

Map → Event

- selected signal enlarges slightly
 - event chamber emerges from its spatial source
 - map recedes into blur/dim
-

Upload Step Progression

- linear horizontal or fade progression
 - progress indicator updates clearly
 - no hard page switch feeling
-

14.10 Micro-Interactions

Important micro-interactions include:

- signal pin hover/tap response
- save icon fill
- filter chip selection
- time pill expansion
- inline edit entry
- field corrected confirmation

These must remain subtle and responsive.

14.11 Duration Principles

Durations should feel:

- fast enough to avoid drag
- long enough to communicate meaning

Guideline:

- micro interactions = short
- mode changes = medium
- confirmation = brief

Avoid sluggishness.

14.12 Easing Principles

Use easing that feels:

- smooth at start
- controlled at finish
- non-elastic

Avoid exaggerated spring behaviors unless extremely restrained.

14.13 Motion Hierarchy

Only one major motion event should dominate at a time.

For example:

- if a modal is opening, do not animate five other surfaces strongly
 - if a save action confirms, do not also trigger global UI movement
-

14.14 Reduced Motion Support

The system should support reduced-motion preferences.

Fallbacks:

- fade instead of slide
 - no blur animation if performance or accessibility demands it
 - shorter transition ranges
-

14.15 Performance Constraints

Motion must not degrade the app.

Critical rules:

- avoid heavy blur on weak devices
 - limit simultaneous glows
 - keep map interaction responsive under overlay transitions
 - prioritize frame stability over decorative effects
-

14.16 Motion Integrity Rule

Every motion must clarify state. If it does not clarify state, remove it.

Closing Statement (Sections 12–14)

These sections make the system buildable:

- **Section 12** defines the reusable component language
- **Section 13** defines the visual identity and readability rules
- **Section 14** defines the behavioral motion logic

At this point, the WeUP Phase 0 UI/UX Bible now has a complete core foundation across:

- state
- layout
- navigation
- time
- map
- calendar

- event detail
 - upload
 - saved
 - profile
 - components
 - visual language
 - motion
-

Next recommended sections

- **15–18** → Data model, monetization surfaces, moderation hooks, analytics
- **19–21** → performance, implementation blueprint, acceptance criteria

WeUP Phase 0 — UI/UX Bible (Sections 15–18)

Focus: Data Model (UX-facing), Monetization Surfaces, Security/Moderation, Analytics

Objective: Bridge UX system → real-world operation → revenue → governance

15. Data & Query Model (UX-Facing)

15.1 Purpose

The data model must support:

- fast map rendering
- accurate event representation
- flexible filtering
- OCR-derived uncertainty
- future monetization hooks

This is not just backend structure.

It directly determines:

what the user sees, how fast it appears, and how trustworthy it feels

15.2 Core Entity: Event

The **event** is the primary unit of the system.

Required Fields

- id
- title
- datetime_start
- venue_name OR location
- geo (lat, lng)

Optional Fields

- datetime_end
- address
- neighborhood
- tags (genre, type)
- description
- flyer_image_url
- source
- created_by
- created_at

System Fields (Hidden from UI)

- ocr_confidence_score
- parse_confidence_score
- dedupe_group_id
- moderation_status
- featured_flag
- boost_score

15.3 Event State Lifecycle

Each event exists in one of these states:

- **Draft** → user-uploaded, not yet confirmed
 - **Parsed** → OCR + AI structured
 - **Published** → visible on map/calendar
 - **Flagged** → under moderation
 - **Merged** → deduplicated with another event
 - **Archived** → expired or removed
-

15.4 Query Model (UX-Driven)

The UI dictates query patterns.

Map Queries

Inputs:

- viewport bounding box
- time range
- optional filters

Output:

- clustered event set
 - lightweight payload
-

Calendar Queries

Inputs:

- selected date
- optional sort

Output:

- grouped event arrays
 - enriched metadata for display
-

Event Detail Query

Input:

- event_id

Output:

- full event object
 - enriched metadata
-

15.5 Geo Query Constraints

To maintain performance:

- use geohash or spatial indexing
 - limit results to visible map area
 - cluster aggressively when density is high
-

15.6 Time Query Model

Time must be:

- indexed
- queryable in ranges
- normalized across formats

Example:

- now → next 4 hours
 - tonight → 6pm–3am
 - weekend → Fri–Sun
-

15.7 Deduplication Model

Events may be duplicated due to multiple uploads.

Deduping logic should consider:

- title similarity
- venue match
- time overlap

UI result:

- user sees one clean event
 - backend merges or groups duplicates
-

15.8 Data Freshness

Events must:

- expire automatically
 - move to archive state
 - stop appearing in queries
-

15.9 Loading States

The data model must support:

- partial loading (map signals first)
 - progressive enrichment (details later)
-

15.10 System Integrity Rule

Data must always resolve into a clean, readable event object for the UI.

16. Monetization Surfaces (Phase 0)

16.1 Philosophy

WeUP monetizes:

visibility and positioning, not transactions (initially)

16.2 Primary Monetization Surfaces

16.2.1 Featured Events

Events marked as featured:

- stronger glow on map
- larger presence in calendar
- priority in clustering

UI must visually distinguish but not overwhelm.

16.2.2 Boosted Upload

During upload:

User can:

- boost event visibility

UI placement:

- after publish confirmation
 - optional upsell
-

16.2.3 Venue Claim

Venues can:

- claim their location
- control event listings
- gain persistent presence

UI:

- “Claim this venue” entry point in event detail or map
-

16.3 Secondary Monetization Surfaces

Sponsored Zones

Map regions can:

- glow differently
- display sponsor attribution

Must remain subtle and non-intrusive.

Calendar Placement

Featured events:

- appear larger
 - appear earlier
 - occupy premium positions
-

16.4 Monetization UI Rules

- no intrusive ads
- no popups blocking interaction
- no banner clutter

Monetization must feel:

- integrated
 - premium
 - optional
-

16.5 Trust Considerations

Users must understand:

- why something is featured
 - that content is not manipulated deceptively
-

16.6 System Integrity Rule

Monetization must never degrade discovery clarity.

17. Security & Moderation (UX Hooks)

17.1 Purpose

Because WeUP is user-generated:

- content must be moderated
 - abuse must be controlled
 - trust must be maintained
-

17.2 Moderation States

Each event may be:

- clean
 - flagged
 - under review
 - removed
-

17.3 Reporting UX

Users must be able to:

- report an event
- flag incorrect info
- flag inappropriate content

Entry points:

- event detail modal
- profile history

17.4 Report Flow

1. User taps “Report”
2. Selects reason
3. Submits
4. receives confirmation

Must be:

- fast
- low friction
- non-invasive

17.5 Content Integrity Signals

Optional UI indicators:

- verified venue
- high-confidence parse
- community-confirmed

These increase trust.

17.6 Abuse Prevention

System should guard against:

- spam uploads
- duplicate flooding
- fake events
- malicious links

UX hooks:

- upload rate limits
- validation feedback
- review flags

17.7 Admin / Curation Layer

Not exposed in Phase 0 UI, but must exist:

- moderation dashboard
- event merge tools
- content removal controls

17.8 User Feedback Loop

If content is removed:

- user should receive feedback
- reason should be understandable

17.9 System Integrity Rule

Trust must be visible, not assumed.

18. Analytics & Instrumentation

18.1 Purpose

Analytics must answer:

- what users explore
- what they save
- what they ignore
- where supply is growing

18.2 Core Event Tracking

Track:

- map view
 - signal tap
 - event detail open
 - save action
 - upload start
 - upload complete
 - calendar usage
-

18.3 Funnel Definitions

Discovery Funnel

Map View → Signal Tap → Event Detail → Save

Upload Funnel

Upload Start → OCR Complete → Edit → Publish

18.4 Key Metrics

Supply Metrics

- uploads per day
 - parsed success rate
 - event density by area
-

Demand Metrics

- event views
 - saves
 - repeat interactions
-

Conversion Metrics

- view → save
 - upload → publish
-

18.5 Heatmap Analytics

Track:

- time-based interaction density
 - geo-based interaction density
 - high-interest zones
-

18.6 Retention Indicators

- users returning within 24 hours
 - saved events revisited
 - repeated uploads
-

18.7 Experimentation Hooks

Enable:

- UI A/B testing
 - signal visualization experiments
 - calendar density variations
-

18.8 Performance Monitoring

Track:

- map load time
- OCR processing time
- query latency
- modal open latency

18.9 Privacy Considerations

- no unnecessary data collection
 - anonymize location where possible
 - transparent usage
-

18.10 System Integrity Rule

Analytics must inform decisions, not clutter the product.

Closing Statement (Sections 15–18)

These sections connect the UI system to reality:

- **Section 15** → defines how data feeds the UI
- **Section 16** → defines how revenue enters the system
- **Section 17** → defines how trust is preserved
- **Section 18** → defines how the system learns and improves

At this stage, the WeUP Phase 0 Bible now fully covers:

- UX architecture
 - visual + motion systems
 - data + backend alignment
 - monetization entry points
 - moderation + trust
 - analytics + growth loops
-

Final Remaining Sections

- **19–21** → Performance, Implementation Blueprint, Acceptance Criteria

These will translate everything into:

directly buildable engineering execution

If you want, I can now:

- convert this entire bible into a **Vue + Firebase file structure + component map**
- or generate a **Google AI Studio megaprompt that builds the entire system end-to-end**

WeUP Phase 0 — UI/UX Bible (Sections 19–21)

Focus: Performance & Constraints, Implementation Blueprint, Acceptance Criteria

Objective: Translate the design system into an executable product standard

19. Performance & Constraints

19.1 Purpose

Performance is not a technical afterthought.

In WeUP, performance directly affects:

- trust
- perceived product quality
- emotional tone
- usability under nightlife conditions

A slow nightlife radar stops feeling alive.

19.2 Primary Performance Targets

Phase 0 should optimize for:

- fast first render
- stable map interaction
- responsive modal transitions

- lightweight event previews
 - acceptable OCR turnaround
 - controlled memory usage on mobile devices
-

19.3 Critical Interaction Budgets

The following interactions must feel immediate or near-immediate:

High-Priority UI Response Targets

- map pan / zoom responsiveness
- signal tap response
- event peek card appearance
- event save feedback
- bottom nav mode switching

These actions must never feel blocked by:

- heavy image rendering
 - oversized overlays
 - synchronous parsing tasks
 - unnecessary re-fetching
-

19.4 Query Budgeting

All data fetches must be scoped to the active UX need.

Map Mode

Only fetch:

- visible map area
- current time range
- lightweight event payloads

Calendar Mode

Only fetch:

- selected date range

- display-ready card data

Event Detail

Fetch:

- full event payload on demand
 - only when entering detail mode
-

19.5 Partial Rendering Strategy

The UI should not wait for complete payload enrichment when not required.

Recommended approach:

Stage 1

- render map surface
- render lightweight signals

Stage 2

- enrich metadata
- load flyer thumbnails
- populate detail data

This preserves velocity.

19.6 Image Constraints

Flyers are one of the biggest performance risks.

Requirements:

- compress on upload
- generate thumbnail variants
- lazy-load large images
- crop predictably for cards
- load full resolution only in detail mode if needed

19.7 Motion Performance Limits

Motion must never compromise interaction.

Avoid:

- too many simultaneous blur layers
- multiple glow animations at scale
- full-screen repaints triggered by minor actions
- stacked animated panels

Recommended:

- use transform and opacity-based transitions
 - reserve blur for focused modal states only
 - minimize persistent animation loops
-

19.8 Map Load Strategy

Map performance must be protected above all else.

Requirements:

- cluster aggressively at density
 - debounce rapid query updates
 - keep signal rendering efficient
 - limit overlay complexity while panning
 - avoid rendering unnecessary labels during active movement
-

19.9 Upload / OCR Performance Model

Upload flow must separate:

- client responsiveness
- backend extraction work

UX expectations:

- upload starts immediately
 - user sees visible progress
 - OCR/parsing status updates clearly
 - user can recover from delay without confusion
-

19.10 Internal Scroll Constraints

Internal scroll is permitted only inside:

- calendar board content region
- event detail content body
- upload step content where necessary
- profile/saved content region if needed

The shell itself must not scroll.

19.11 Offline / Weak Network Considerations

Phase 0 should tolerate poor mobile conditions.

Recommended support:

- stale cached events for recent sessions
 - graceful empty states
 - upload retry support
 - skeleton loading instead of blank surfaces
-

19.12 Battery / Device Constraints

Because this is a mobile-first nightlife product:

- constant animations must be limited
 - GPS or geolocation should be used carefully
 - maps should avoid wasteful redraw cycles
 - dark mode must remain efficient, not just aesthetic
-

19.13 Performance Integrity Rule

The app must always feel lighter than its visual richness suggests.

20. Implementation Blueprint

20.1 Purpose

This section turns the UI/UX system into a build-ready architecture.

It defines:

- engineering surface boundaries
 - component grouping
 - state ownership
 - data flow expectations
 - implementation discipline
-

20.2 Recommended Frontend Architecture

A practical Phase 0 frontend should use:

- **AppShell**
- **mode-based routing or finite mode state**
- **centralized state store**
- **strict modal manager**
- **shared UI token system**

Recommended major implementation layers:

App Shell Layer

Mode State Layer

Data Query Layer

Map / Calendar / Detail Surface Layer

Feedback / Overlay Layer

20.3 Recommended Mode State Model

The app should use an explicit state model, not ad hoc booleans.

Suggested top-level modes:

- map
- calendar
- eventDetail
- upload
- saved
- profile

Suggested supporting UI state:

- selectedEventId
 - activeTimeWindow
 - activeFilters
 - uploadStep
 - modalStackType
 - mapViewport
 - saveStatusQueue
-

20.4 Recommended File / Component Structure

Example high-level frontend structure:

```
/src
/app
  AppShell
  ModeRouter
  ModalHost
  NavigationController

/components
  /shell
  /map
  /calendar
  /event
  /upload
  /saved
  /profile
```

/feedback

/stores

modeStore

eventStore

mapStore

uploadStore

userStore

/services

eventService

uploadService

mapQueryService

analyticsService

moderationService

/styles

tokens

motion

typography

materials

20.5 App Shell Responsibilities

The shell must own:

- viewport containment
- safe-area handling
- top HUD rendering
- bottom nav rendering
- mode switching
- modal root
- global feedback placement

The shell must not own:

- event logic
 - upload parsing logic
 - map clustering logic directly
-

20.6 Store / State Responsibilities

modeStore

Owns:

- active mode
- prior mode
- transition intent
- visibility contract enforcement

eventStore

Owns:

- event query results
- selected event
- save states
- event detail cache

mapStore

Owns:

- viewport
- bounds
- map density/clustering state
- active time window

uploadStore

Owns:

- upload file
- OCR result
- parsed fields
- edit states
- publish lifecycle

userStore

Owns:

- profile
- preferences

- saved events
 - upload history
-

20.7 UI Composition Blueprint by Mode

Map Mode

Composed of:

- AppShell
 - TopHUD
 - MapCanvas
 - SignalLayer
 - EventPeekCard
 - TimePill
 - BottomNav
-

Calendar Mode

Composed of:

- AppShell
 - ModalLayer
 - CalendarBoard
 - DateSelector
 - CalendarCards
 - LocalCloseControl
-

Event Detail Mode

Composed of:

- AppShell
- ModalLayer
- EventDetailModal
- EventHero
- MetadataBlocks
- CTACluster

Upload Mode

Composed of:

- AppShell
- ModalLayer or dedicated focus surface
- UploadWizard
- OCRStatusPanel
- ParsedFields
- InlineEditFields
- PublishConfirmation

Saved Mode

Composed of:

- AppShell
- SavedHeader
- SavedGrid
- SavedCard set
- EmptyStatePanel

Profile Mode

Composed of:

- AppShell
- ProfileHeader
- PreferenceGroups
- ActivityList
- AccountActions

20.8 Backend-Aligned Service Blueprint

The frontend should assume the following service domains:

Event Service

- query visible events
- fetch event detail
- save/unsave event
- featured event retrieval

Upload Service

- upload flyer
- get OCR status
- update parsed fields
- publish event

Map Query Service

- viewport query
- time range query
- cluster-ready formatting

Analytics Service

- track interactions
- funnel events
- performance instrumentation

Moderation Service

- report content
- fetch moderation state where needed

20.9 Design Token Blueprint

A token layer should be created for:

- colors
- spacing
- border radius
- blur values
- elevation
- typography sizes
- motion durations
- z-index/layer constants

No arbitrary hard-coded design values should be scattered through components.

20.10 Visibility Contract Implementation

The visibility rules defined earlier must be encoded as logic, not left to component accident.

Examples:

- time control only renders in map mode
 - bottom nav suppresses in event detail and upload
 - global filters do not render in profile
 - modal host controls layer priority centrally
-

20.11 Analytics Hook Blueprint

Track at component boundaries, not randomly.

Examples:

- signal tap from SignalPin
 - event detail open from EventDetailModal mount
 - upload completion from UploadWizard final state
 - save action from CTACluster
 - calendar tile open from CalendarCard
-

20.12 Implementation Integrity Rule

Every mode, component, and service must map cleanly to the state system. No orphan UI logic.

21. Phase 0 Acceptance Criteria

21.1 Purpose

Acceptance criteria define when Phase 0 is operationally and experientially valid.

This is not only about whether features exist.

It is about whether the system behaves correctly.

21.2 Core Product Acceptance

Phase 0 is accepted only if the following loop works end-to-end:

Upload flyer → OCR/parse → publish event → event appears on map/calendar → user opens detail → user saves

If this loop breaks, Phase 0 is incomplete.

21.3 Viewport Compliance

The product passes only if:

- no page-level vertical scrolling exists
 - all major modes fit inside one viewport
 - internal scrolling is limited to designated surfaces only
 - no overlay extends beyond device-safe bounds
-

21.4 Mode Clarity

The product passes only if:

- one dominant mode is always obvious
 - users can identify current mode immediately
 - transitions between modes are visually coherent
 - no simultaneous primary surfaces compete
-

21.5 Time System Compliance

The product passes only if:

- time is contextual, not global
 - time controls are hidden outside relevant modes
 - changing time clearly affects map results
 - time UI does not collide with navigation
-

21.6 Navigation Compliance

The product passes only if:

- bottom nav does not compete with modals
 - close/back behavior is consistent
 - deep states suppress irrelevant global controls
 - users can always recover to map state predictably
-

21.7 Map Experience Compliance

The product passes only if:

- map remains visually dominant in map mode
 - signal density remains readable
 - peek cards stay compact
 - interaction remains stable while panning and zooming
-

21.8 Calendar Compliance

The product passes only if:

- calendar behaves as a true takeover surface
 - calendar uses modular or masonry-like event composition
 - event selection from calendar transitions clearly into detail mode
 - calendar content remains readable under density
-

21.9 Event Detail Compliance

The product passes only if:

- event detail feels like a focused chamber
 - unrelated controls disappear
 - CTAs are clear and prioritized
 - missing or weak OCR data does not break readability
-

21.10 Upload Flow Compliance

The product passes only if:

- upload flow is step-based and comprehensible
 - OCR status is visible
 - parsed data is editable inline
 - manual correction is easy
 - user can publish even if OCR is imperfect
-

21.11 Saved & Profile Compliance

The product passes only if:

- saved mode feels intentional and useful
 - profile mode is spacious and uncluttered
 - both modes remove irrelevant controls like time slider
 - cross-mode return behavior is stable
-

21.12 Visual System Compliance

The product passes only if:

- the dark + neon brand is preserved
 - readability remains strong
 - depth is clear
 - visual clutter is reduced from the current MVP
-

21.13 Motion Compliance

The product passes only if:

- transitions clarify state
 - no major motion causes confusion
 - motion remains performant on mobile
 - modal entry/exit preserves spatial continuity
-

21.14 Performance Compliance

The product passes only if:

- map interactions feel responsive
 - signal taps respond quickly
 - large flyer media does not stall the UI
 - OCR pipeline communicates delay clearly
 - app remains usable on mid-tier mobile devices
-

21.15 Trust & Moderation Compliance

The product passes only if:

- users can report problematic content
 - duplicate or false content can be managed
 - monetized/featured content is visually distinguishable
 - platform trust signals are present where needed
-

21.16 Analytics Compliance

The product passes only if:

- core discovery funnel events are tracked
- upload funnel is measurable
- save behavior is measurable
- performance events are instrumented

21.17 Phase 0 Completion Statement

Phase 0 is complete when:

1. The supply loop works
2. The discovery loop works
3. The save loop works
4. The interface is spatially clear
5. The product feels premium and stable

21.18 Final Integrity Rule

A feature is not complete when it is visible.

It is complete when it behaves correctly within the system.

Closing Statement (Sections 19–21)

These final sections turn the WeUP Phase 0 UI/UX Bible into an execution-ready product document.

The bible now defines:

- philosophy
- state architecture
- layout rules
- navigation
- time logic
- map/calendar/detail behavior
- upload/saved/profile behavior
- component language
- visual identity
- motion logic
- data and monetization
- moderation and analytics
- performance rules
- implementation structure

- acceptance criteria

This is now sufficient to drive:

- frontend build
- Firebase architecture
- AI Studio prompting
- QA validation
- investor/product communication

A sensible next step is to convert this into a **condensed formal whitepaper PDF structure** or a **developer implementation spec with components, routes, stores, and Firebase collections**.